Unsupervised Machine Learning

- It is a type of ML where a model learns patterns and relationships within input data without explicit labels or pre-defined outputs / outcomes.
- Unlike supervised Learning where the model learns from labeled data examples, unsupervised learning explores hidden structured and meaningful groups within data without human provided labels.
- This approach is widely used when dealing with ~~ex~~ complex and unstructured data.

Application

- Clustering
- Dimensionality reduction
- Anomaly detection
- Recommendation system (Filters, Collaborative & hybrid)
- Image compression and Generation

Types :-

Unsupervised Learning

Clustering	Dimentionality Reduction	Anomaly detection	Association rule mining
k-means	PCA	Isolation forest	Apriori
k-medoids	t-SNE	One class SVM	FP Growth
Hierarchical	SVD		
→ Agglomerative			
→ Divisive			
Density based			
DBSCAN			
Density & distance			

__/__/__

Advantages

- No need for labeled data :- Useful for extracting insights from vast amount of unlabeled data which can be more readily available than labeled datasets.

- Data Exploration :- Effective for discovering hidden pattern, correlations and structures.

- Dimensionality Reduction :- Help simplify data while retaining meaningful features for downstream analysis.

Disadvantages

- lack of Interpretability :- The result and insights obtained may be harder to interpret without explicit labels.

- Complexity :- Algorithm may struggle with very large, high-dimensional datasets without appropriate preprocessing.

- Risk of Overfiting :- Without the guidance of labels, models may find irrelevant pattern that don't generalize well.

Curse of dimensionality 25/11/24

- The curse of dimensionality in ML arises when working with Higher dimensional data which leads to increased computational complexity and overfitting
- To deal with this problem we use techniques like dimensionality reduction, feature selection and careful model design are essential for mitigating its effect and improving algorithms performance

- Navigating this challenge is crucial for unlocking the potential of High-dimensional dataset and ensuring about robust ML solution

How to overcome the curse of dimensionality

Dimensionality Reduction Technique :

- It is a way of converting higher dimension data into lesser dimension dataset ensuring that it provides similar information.

a) Feature Selection :- It is the process of selecting the most relevant feature from the whole dataset to provide appropriate result.

b) Feature extraction :- It is the process of extracting, transforming raw data into a set of relevant and meaningful features for analysis by combining or mapping existing features.

Benefits of applying Dimensionality reduction

- By reducing the dimensions of the features, the space required to store the dataset also gets reduced
- less computation training time is required for ~~d~~ reducing dimensions of features.
- Accuracy improvements
- overfitting risk reduction
- Speed up in training
- improved data visualization
- Increase in explainability of our model.
- Reduced dimensions of features of the datset help in visualizing the data quickly.
- It removes the redundant feature (if present) by taking care of multicollinearity.

Disadvantages

- While doing dimensionality reduction, we lost some of the info" which can possibly affect the performance of subsequent training algorithms.
- It can be computationally intensive
- Transformed features are often hard to interpret
- It makes the independent variables less interpretable.

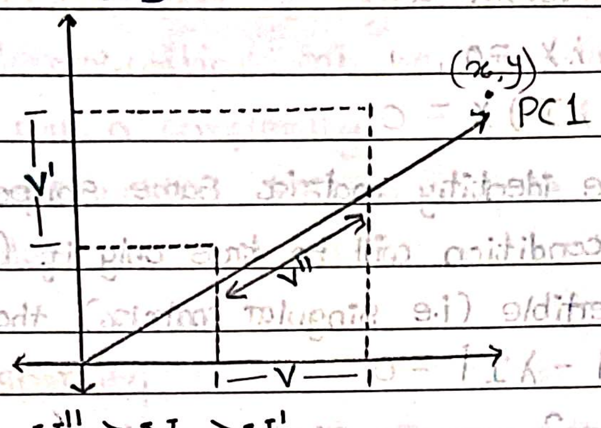
28/11/24

Principal Component analysis (1901)

- PCA technique was introduced by the mathematician Karl Pearson in 1901. It works on the condition that while the data in the higher dimensional space is mapped to the data in the lower dimensional space, the ~~variat~~ variance of the data in the lower dimensional space should be maximum
- PCA is a statistical procedure that uses an orthogonal transformation that converts a set of co-related variables to a set of ~~un~~ uncorelated variables
- PCA is most widely used tool for visualization of higher dimensional data in ML and also used for predictive models.
- PCA is an unsupervised learning algorithm used to examine the inter-relations among a set of variables. It is also known as 'general factor analysis'
- The main goal of PCA is to reduce the dimension of a dataset while preserving the most important patterns or relationship between the variables without any prior knowledge of the target variable.
- PCA reduce the dimensionality of dataset by finding a newset of variables, smaller than the original set of variables, retaining most of the sample information and can also be used for regression & classification

5/12/24

2D → 1D



$V'' > V > V'$

Steps of PCA :-

1) Standardization
- First we need to standardize our dataset to ensure that each variable has mean of '0' and standard deviation '1'

$$z = \frac{x_i - \mu}{\sigma}$$

2) Calculate Covariance matrix
- Covariance measures the strength of joint variability between two or more variables, indicating how much they change in relation to each other
- To find the covariance we can use the formula

$$cov(x, y) = \frac{\Sigma (x-\bar{x})(y-\bar{y})}{N-1}$$

3) Compute eigen vectors and values of cov. matrix to identify principal component
- let A be a square of nxn matrix and x be a non-zero vector for which

$$AX = \lambda X$$

for some scalars values 'λ', then 'λ' is known as eigen values of matrix 'A' and 'X' is known as eigen vectors of a matrix A

__/__/__

- It can be also return as

$$AX - \lambda X = 0$$
$$(A - \lambda I) X = 0$$

Where, I is the identity matrix same shape as matrix 'A'. and the above condition will be true only if $(A - \lambda I)$ will be non-negative invertible (i.e singular matrix) that means

$$|A - \lambda I| = 0$$

from the above eq^n we can find the eigen values λ and then corresponding vectors can be formed using eq^n

$$AX = \lambda X$$

4) Projecting every data points to eigen vectors

- Eventually we are going to project all the data point to our eigen vectors to create principal components

Computational issues with PCA

i) High Dimensionality of Data :

- When the number of features is very large, computing cov. matrix and it's eigen values can be computationally expensive
- Solution :- use singular value decomposition (SVD) instead of directly calculating the covariance matrix, SVD scales better with high-dimensional data.

ii) Numerical Stability

- The cov. matrix can be ill-conditioned specially, when features are highly corelated. This can lead to numerical instability in eigen value.
- Sol^n :- standardize the data before applying PCA (mean = 0, variance = 1)
- use regularized PCA, adding a small constant to diagonal of cov. matrix

__/__/__

iii) Computational cost of eigen decomposition

- Eigen decomposition can be computationaly expensive for large matrices with a complexity $O(n^3)$ for nxn matrices
- Sol^n:- Use truncated SVD to compute only the top 'k' principal components.

iv) Data sparsity

- Sparse datasets such as those encountered in text mining or recommendation system can lead to insufficient standard PCA implementation.
- Sol^n:- Use sparse PCA we presere sparsity in principal components

v) Interpretation of results

- The result of PCA can be difficult to interpret, especially in high-dimensional spaces
- Sol^n:- Visualize the first few components and use technique like varimax rotation to make the components movre interpretable

vi) Missing Data

- PCA assumes a complete dataset. Missing values can cause the covariance matrix computation to fail
- Sol^n:- Impute missing values using techniques like mean, median or k-nearest neighbours (KNN) imputation
- Use robust PCA methods that can handle missing data directly

vii) Choices of Number of components

- Determining the optimal number of Principal components can be challenging
- Sol^n:- Use explained variance ratio to decide the number of components that capture a desired percentage of variance.
- Apply cross-validation to evaluate the performance of different number of components

__/__/__

viii) Overfitting in Small datasets

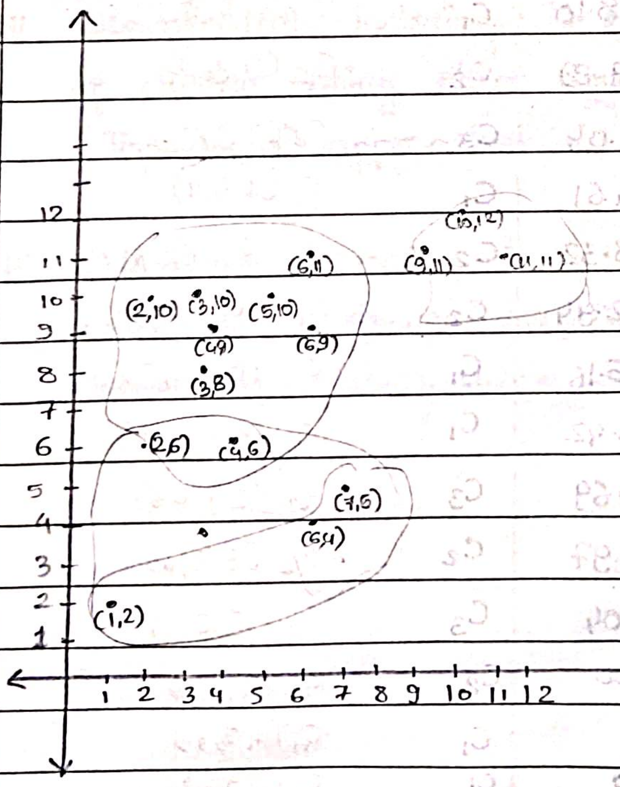
- using too many principal components in small datasets can lead to overfitting

Sol^n :- limit the number of components by considering domain knowledge and the elbow method in the Scree plot

12/12/24

K means clustering

		C_1	C_2	C_3					
x	y	3,8	7,5	11,11		3.6,8.1	6.5,4.5	10,11.3	
2	10	2.24	7.07	9.06	C_1	2.48	7.10	8.10	C_1
2	6	2.24	5.1	10.3	C_1	2.64	4.7	9.59	C_1
11	11	8.5	7.2	0	C_3	7.94	7.91	1.04	C_3
6	9	3.16	4.12	5.39	C_1	2.5	4.5	4.61	C_1
6	4	5	1.41	86	C_2	4.75	0.70	8.32	C_2
1	2	6.3	6.7	13.45	C_1	6.63	6.04	12.9	C_2
5	10	2.82	5.38	6.08	C_1	2.36	5.7	5.16	C_1
4	9	1.41	5	7.28	C_1	0.98	5.14	6.42	C_1
10	12	8.06	7.61	1.41	C_3	7.49	8.27	0.69	C_3
7	5	5	0	7.21	C_2	4.6	0.70	6.9	C_2
9	11	6.7	6.3	2	C_3	6.12	6.96	1.04	C_3
4	6	2.23	3.16	8.68	C_1	2.13	2.9	8.0	C_1
3	10	2	6.40	8.06	C_1	1.99	6.5	7.11	C_1
3	8	0	5	8.54	C_1	0.60	4.94	7.73	C_1
6	11	4.24	6.08	5	C_1	3.76	6.51	4.011	C_1



$C_1 = \frac{36}{10} = 3.6$, $\frac{81}{10} = 8.6$

$C_2 = 6.5, 4.5$

$C_3 = 10, 11.3$

$C_1 = 3.88, 8.77$

$C_2 = 4.66, 3.66$

$C_3 = 10, 11.33$

3.87, 9.125   4 , 4.6
4.5 , 4.25
10 , 11.33   10, 11.33

x	y	(3.88, 8.77)	4.66, 3.66	10, 11.33	
2	10	2.24	6.87	8.10	C_1
2	6	3.34	3.5	9.59	C_1
11	11	7.46	9.69	1.04	C_3
6	9	2.13	5.5	4.61	C_1
6	4	5.21	1.38	8.32	C_2
1	2	7.35	4.01	12.94	C_2
5	10	1.66	6.34	5.16	C_1
4	9	6.25	5.38	6.42	C_1
10	12	6.92	9.9	0.69	C_3
7	5	4.8	2.6	6.97	C_2
9	11	5.5	8.5	1.04	C_3
4	6	2.77	2.43	8.00	C_2
3	10	1.51	6.55	7.11	C_1
3	8	1.16	4.64	7.73	C_1
6	11	3.07	7.46	4.01	C_1

3.87, 9.125	4.5, 4.25	10, 11.33		
2.06	6.2	~~C_1~~ 8.10	C_1	C_1 (2,5,8)
3.64	3.05	~~C_2~~ 9.59	C_2	C_2 (4,2,9)
7.37	9.37	~~C_1~~ 1.04	C_3	C_3 (7,9,1)
2.13	4.98	~~C_1~~ 4.61	C_1	C_1 (1,4,4)
5.55	1.52	~~C_2~~ 8.32	C_2	C_2 (5,2,8)
7.68	4.16	~~C_2~~ 12.94	C_2	C_2 (8,3,12)
1.42	5.77	~~C_1~~ 5.16	C_1	C_1 (0.9,5,5)
0.18	4.77	~~C_1~~ 6.42	C_1	C_1 (0.4, 4,6)
6.77	9.5	~~C_1~~ 0.69	C_3	C_3 (6,9, 0.6)
5.17	2.61	~~C_2~~ 6.97	C_2	C_2 (5,3,6)
5.46	8.11	1.04	C_3	C_3 (5,8,1)
3.12	1.82	8.00	C_2	C_2 (3,1,8)
1.23	5.94	7.11	C_1	C_1 (1,5,7)
1.42	4.03	7.73	C_1	C_1 (1,3,7)
[illegible]	6.9	4.01	C_1	C_1 (2,6,4)

16/12/24

Kmeans

What is clustering ?

- clustering is an unsupervised ML technique i.e used to group similar data points together based on certain characteristics or patterns
- It aims to identify natural grouping or clusters in a dataset where datapoints with in the same cluster are more similar to each other then to those in other cluster.
- Since clustering is an unsupervised Learning technique it does not require label data.
- To calculate similarities among the datapoints it uses similarity measure's such as euclidean, Manhattan, distances and cosine similarity

Common clustering Algorithm

i. K-means clustering : It divies the data into 'k' predefined clusters using distance metrics, and minimize the variance within the cluster

ii. Hierarchical clustering : It builds a hierarchy (Tree like structure) of clusters using either agglomerative (bottom-up) or Divisive (Top-down) approaches.

iii. DBSCAN (Density-Based-Spatial-Clustering of Application with Noise) : Groups datapoints based on density, Identifying clusters of arbitary shape.

__/__/__

Applications of clustering

i. Customer segmentation: Grouping customer based on purchasing behaviour.

ii. Image segmentation: Partioti oning images in to the region for better analysis

iii. Anomaly detection: Identifying outlier in the data.

iv. Document categorization: Organizing data into topic

K-means clustering is one of the most popular unsupervised clustering algo^n. It partitions a D.S in 'k' predefined clusters, where each cluster contains datapoints that are similar to each other & different from points in other cluster.

K-means clustering is a centriod based algorithm or a distance based algo^n where we calculate the daristance to assign a point to a cluster. In K-means each cluster is associated with a centriod.

Algorithm

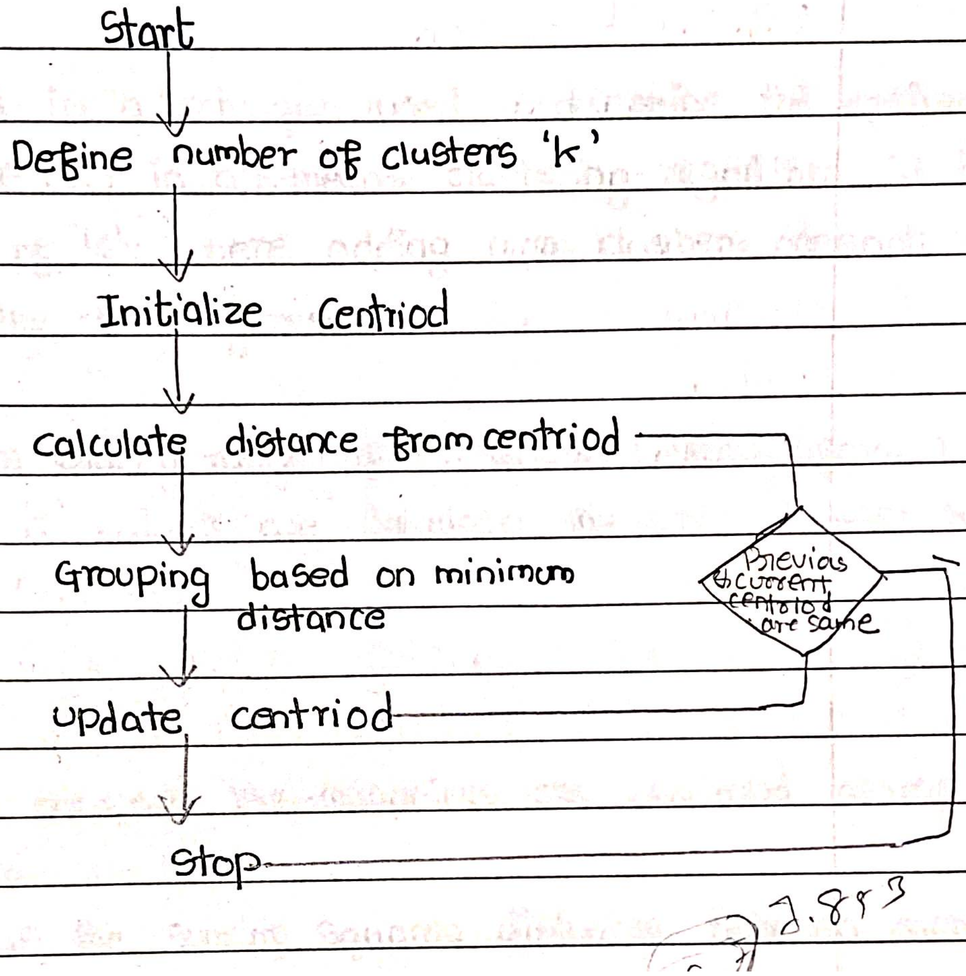
Step 1: Define value of 'k', firstly we pass the value of predefine clusters K.

Step 2: Initialize centsiods, Randomly select 'k' points as the initial cluster centriod.

Step 3: Assign clusters, for each data point calculate its distance (e.g. euclidean distance) to each centriod & assign it to the nearest cluster.

Step 4: Update centriod, calculate the mean of all datapoints in each cluster & update the centriod

Step 5: ~~Repeat~~ ~~se~~ Itterate; repeat step 3 & 4 untill the centriod no longer change significantly or a stoping condition (maximum iteration) is met.



Start
Define number of clusters 'k'
Initialize Centriod
calculate distance from centriod
Grouping based on minimum distance
update centriod
Stop
Previas & current centriod are same

7.893

WCSS
Sum.inertia = WCSS

x_1, x_2

17/12/24

Elbow Method

- The elbow method is a technique used to determine the optimal number of cluster (k) in a k-means clustering algorithm. It helps to find the value of 'k' where adding more clusters doesnot significantly improve the clustering result.

Method

step 1: We iterate over a range of 'k' values typically from 1 to N

step 2: For each 'k' values we calculate the within cluster sum of square (WCSS)

WCSS

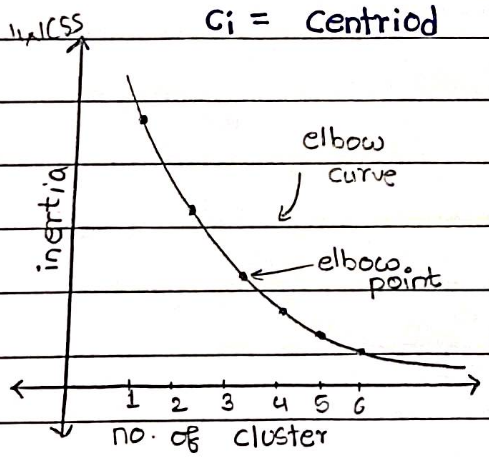
WCSS measures how well the datapoints are clustered around there respective centriod

- It is defined as the sum of squared distances between each point and it's cluster centriod.

$$WCSS = \sum_{i=1}^{k} \sum_{j=1}^{n} \text{distance}(x_j, c_i)^2$$

where x_i = distance point

c_i = centriod



Elbow Curve

The score ranges from -1 to +1
Points are very close to there own cluster & far from others (Ideal clustering)
0 : Points are on or near the decision boundary between clusters
-1 : Points are assign to the wrong cluster

Silhouette score

The silhouette score is a metric used to evaluate the quality of clustering results. It measures how similar a point is to its own cluster compared to other cluster.

A higher score indicate, the clusters are well seperated and cohesive

Formula: The silhouette score for a single sample 'i' is

$$S(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

where $a(i)$ = The average intra-cluster distance (distance between point 'i' & other points in the same cluster)

$b(i)$ = The avg. nearest-cluster distance (distance between point 'i' & points in the nearest neighbouring cluster).